

day1 讲义

Equifix

2026 年 4 月 30 日

题意：给你一个数组 x_i ，让你将其中某一个数加上 X ，问所有前缀和均非负的非空区间数量最大值。

$n \leq 5 \times 10^5$ 。

- 首先将原序列前缀和，设前缀和数组为 s 。题意即为选择 (l, r) 使得 $\forall l \leq k \leq r, s_k \geq s_{l-1}$ 。

- 首先将原序列前缀和，设前缀和数组为 s 。题意即为选择 (l, r) 使得 $\forall l \leq k \leq r, s_k \geq s_{l-1}$ 。
- 考虑对每个位置 $0 \leq i \leq n$ ，找到其右侧第一个比它小的位置，设其为 r_i ，即 $r_i = \min_{j>i, s_j<s_i} j$ 。若不存在这样的 j ，则 $r_i = n + 1$ 。

- 首先将原序列前缀和，设前缀和数组为 s 。题意即为选择 (l, r) 使得 $\forall l \leq k \leq r, s_k \geq s_{l-1}$ 。
- 考虑对每个位置 $0 \leq i \leq n$ ，找到其右侧第一个比它小的位置，设其为 r_i ，即 $r_i = \min_{j>i, s_j<s_i} j$ 。若不存在这样的 j ，则 $r_i = n + 1$ 。
- 不难发现对于确定的 s ，答案就是 $\sum_{i=0}^n (r_i - i) = \sum_{i=0}^n r_i - \frac{n(n+1)}{2}$ 。

- 考虑增量 X 的影响。它对前缀和数组的影响就是一段后缀加上 X 。对 $X \geq 0$ 和 $X < 0$ 分开考虑：

- 考虑增量 X 的影响。它对前缀和数组的影响就是一段后缀加上 X 。对 $X \geq 0$ 和 $X < 0$ 分开考虑：
- $X \geq 0$ 时，假设其修改的位置为 $1 \leq i \leq n$ 。对于一个位置 j ，若 $j < i \leq r_j$ ，则变化后的 r_j 可能发生变化至 r'_j ，即若 $j < i \leq r_j$ 则最终答案将增大 $r'_j - r_j$ ；否则，显然 r_j 不发生变化。

- 考虑增量 X 的影响。它对前缀和数组的影响就是一段后缀加上 X 。对 $X \geq 0$ 和 $X < 0$ 分开考虑：
- $X \geq 0$ 时，假设其修改的位置为 $1 \leq i \leq n$ 。对于一个位置 j ，若 $j < i \leq r_j$ ，则变化后的 r_j 可能发生变化至 r'_j ，即若 $j < i \leq r_j$ 则最终答案将增大 $r'_j - r_j$ ；否则，显然 r_j 不发生变化。
- r'_j 的求值非常简单，由于 $r_j \sim r_{r_j} - 1$ 这一段的所有数都不比 r_j 这个位置小，所以有可能成为 r'_j 的只有可能是按 r 往后跳得到的位置，倍增一下即可。将所有这样的区间都求出来，我们可以得到当 i 取每一个位置时答案的增大量，直接取最大值就是答案。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。
- 但我们发现：如果存在 $s_i < 0$ ，我们直接选择第一个最小的 s_i 就能做到理论最优了（答案不变）。但也是有可能所有 s_i 都非负的。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。
- 但我们发现：如果存在 $s_i < 0$ ，我们直接选择第一个最小的 s_i 就能做到理论最优了（答案不变）。但也是有可能所有 s_i 都非负的。
- 注意到我们不能选择 $i = 0$ ，所以如果 $s_0 = 0$ 成为最小值就尴尬了。但其实这启示我们考察 s_i 中所有的与后缀最小值相等的位置。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。
- 但我们发现：如果存在 $s_i < 0$ ，我们直接选择第一个最小的 s_i 就能做到理论最优了（答案不变）。但也是有可能所有 s_i 都非负的。
- 注意到我们不能选择 $i = 0$ ，所以如果 $s_0 = 0$ 成为最小值就尴尬了。但其实这启示我们考察 s_i 中所有的与后缀最小值相等的位置。
- 如果我们选择的 i 不是后缀最小值，我们断言它不是最优的。设 i 后面第一个后缀最小值位置是 k ，则令 $i = k$ 肯定更优。这是因为，所有 $j < i$ 的位置要么本来就不变，要么会变化到一个 $i \sim k$ 之间的值，因为 k 是后缀最小值；对于 $i \leq j < k$ ，其 r 本来就不超过 k ；对于 $k \leq j$ ，两种选法都没变化。所以将 i 设为 k 只会导致 r 变大。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。
- 但我们发现：如果存在 $s_i < 0$ ，我们直接选择第一个最小的 s_i 就能做到理论最优了（答案不变）。但也是有可能所有 s_i 都非负的。
- 注意到我们不能选择 $i = 0$ ，所以如果 $s_0 = 0$ 成为最小值就尴尬了。但其实这启示我们考察 s_i 中所有的与后缀最小值相等的位置。
- 如果我们选择的 i 不是后缀最小值，我们断言它不是最优的。设 i 后面第一个后缀最小值位置是 k ，则令 $i = k$ 肯定更优。这是因为，所有 $j < i$ 的位置要么本来就不变，要么会变化到一个 $i \sim k$ 之间的值，因为 k 是后缀最小值；对于 $i \leq j < k$ ，其 r 本来就不超过 k ；对于 $k \leq j$ ，两种选法都没变化。所以将 i 设为 k 只会导致 r 变大。
- 由此，我们只会选择后缀最小值进行操作，所以可能变化的 r 也就是所有后缀最小值的一个区间，双指针维护一下即可。

股票劝说

- $X < 0$ 时，前面的方法用不了了，因为变化区间变得不像之前那么简单了。
- 但我们发现：如果存在 $s_i < 0$ ，我们直接选择第一个最小的 s_i 就能做到理论最优了（答案不变）。但也是有可能所有 s_i 都非负的。
- 注意到我们不能选择 $i = 0$ ，所以如果 $s_0 = 0$ 成为最小值就尴尬了。但其实这启示我们考察 s_i 中所有的与后缀最小值相等的位置。
- 如果我们选择的 i 不是后缀最小值，我们断言它不是最优的。设 i 后面第一个后缀最小值位置是 k ，则令 $i = k$ 肯定更优。这是因为，所有 $j < i$ 的位置要么本来就不变，要么会变化到一个 $i \sim k$ 之间的值，因为 k 是后缀最小值；对于 $i \leq j < k$ ，其 r 本来就不超过 k ；对于 $k \leq j$ ，两种选法都没变化。所以将 i 设为 k 只会导致 r 变大。
- 由此，我们只会选择后缀最小值进行操作，所以可能变化的 r 也就是所有后缀最小值的一个区间，双指针维护一下即可。
- $X \geq 0$ 时时间复杂度为 $O(n \log n)$ ， $X < 0$ 时时间复杂度为 $O(n)$ 。

逃生游戏

题意：给你一棵有根树，点有点权 x_i ，要你选出一个集合的点，使得根 1 到每个叶子的路径上都恰有 k 个集合内的点，且这些点按顺序构成一个非升序列。对 $x_i \in [L_i, R_i]$ 的所有情况求方案数之和。
 $n \leq 200, k \leq 20$ 。

- 这题相对比较传统。对没有见过类似题目的人来讲可能会很新颖，见过的那就可以秒杀。btw 模数不是让你 NTT 的（

逃生游戏

- 这题相对比较传统。对没有见过类似题目的人来讲可能会很新颖，见过的那就可以秒杀。btw 模数不是让你 NTT 的（
- 考虑一个简单的 dp: $f_{i,j,T}$ 表示已经考虑了 i 的子树，现在从 i 到子树内每个叶子的路径上都选了恰好 j 个点，且现在离根相对最近的几个点的点权最大值是 T 。这个 dp 直接做是 $O(nkR)$ 的，有个 R 显然无法接受。

逃生游戏

- 这题相对比较传统。对没有见过类似题目的人来讲可能会很新颖，见过的那就可以秒杀。btw 模数不是让你 NTT 的（
- 考虑一个简单的 dp: $f_{i,j,T}$ 表示已经考虑了 i 的子树，现在从 i 到子树内每个叶子的路径上都选了恰好 j 个点，且现在离根相对最近的几个点的点权最大值是 T 。这个 dp 直接做是 $O(nkR)$ 的，有个 R 显然无法接受。
- 显然要对这玩意前缀和一下，后面我们就用 f 代表上面那个 dp 的前缀和。

逃生游戏

- 这题相对比较传统。对没有见过类似题目的人来讲可能会很新颖，见过的那就可以秒杀。btw 模数不是让你 NTT 的（
- 考虑一个简单的 dp: $f_{i,j,T}$ 表示已经考虑了 i 的子树，现在从 i 到子树内每个叶子的路径上都选了恰好 j 个点，且现在离根相对最近的几个点的点权最大值是 T 。这个 dp 直接做是 $O(nkR)$ 的，有个 R 显然无法接受。
- 显然要对这玩意前缀和一下，后面我们就用 f 代表上面那个 dp 的前缀和。
- 我们把式子写出来：

$$f_{i,j,T} = (R_i - L_i + 1) \prod_v f_{v,j,T} + [L_i \leq T \leq R_i] \prod_v f_{v,j-1,T}.$$

- $$f_{i,j,T} = (R_i - L_i + 1) \prod_v f_{v,j,T} + [L_i \leq T \leq R_i] \prod_v f_{v,j-1,T}$$

逃生游戏

- $f_{i,j,T} = (R_i - L_i + 1) \prod_v f_{v,j,T} + [L_i \leq T \leq R_i] \prod_v f_{v,j-1,T}$
- 考虑部分分 $L = 1, R = 10^9$ 。我们发现上面的 dp 式子的中括号没了，剩下的都是一顿乘和加的操作。如果我们把这玩意换一个写法呢？

逃生游戏

- $f_{i,j,T} = (R_i - L_i + 1) \prod_v f_{v,j,T} + [L_i \leq T \leq R_i] \prod_v f_{v,j-1,T}$
- 考虑部分分 $L = 1, R = 10^9$ 。我们发现上面的 dp 式子的中括号没了，剩下的都是一顿乘和加的操作。如果我们把这玩意换一个写法呢？
- 设 $F_{i,j}$ 是一个关于 T 的函数，其有 $F_{i,j}(T) = f_{i,j,T}$ 。上式可改为 $F_{i,j} = (R_i - L_i + 1) \prod_v F_{v,j} + \prod_v F_{v,j-1}$ 。而注意到初值为叶子的 $F_{i,0} = R_i - L_i + 1$ 为定值， $F_{i,1} = T - L_i + 1$ 是一次函数，很容易发现的事情是 $F_{i,j}$ 是一个关于 T 不超过 siz_i 次的多项式，所以我们只要求出所有 $T \leq n + 1$ 的答案，就可以利用拉格朗日插值将 $T = R$ 的答案算出来。

逃生游戏

- 回到原问题，由于中括号的存在，给这个函数附加了一个条件判断运算，这样就不是多项式可以接受的乘、加运算了。

逃生游戏

- 回到原问题，由于中括号的存在，给这个函数附加了一个条件判断运算，这样就不是多项式可以接受的乘、加运算了。
- 不过我们发现，如果 T 在一个区间 $[l, r]$ 内，使得每个中括号都是常量，那么刚才的算法就又对了。

逃生游戏

- 回到原问题，由于中括号的存在，给这个函数附加了一个条件判断运算，这样就不是多项式可以接受的乘、加运算了。
- 不过我们发现，如果 T 在一个区间 $[l, r]$ 内，使得每个中括号都是常量，那么刚才的算法就又对了。
- 于是，我们将 $L_i, R_i + 1$ 放在一起离散化，这样得到的每个区间 $x_i, x_{i+1} - 1$ 就满足了我们刚刚说的情况。这时候，我们将 $f_{i,j}(x_i - 1)$ 视为已知定值，则该区间内多项式的次数仍然满足原来的性质，依旧计算 $n + 1$ 个位置的值后将区间右端点插出来，循环做就做完了。

逃生游戏

- 回到原问题，由于中括号的存在，给这个函数附加了一个条件判断运算，这样就不是多项式可以接受的乘、加运算了。
- 不过我们发现，如果 T 在一个区间 $[l, r]$ 内，使得每个中括号都是常量，那么刚才的算法就又对了。
- 于是，我们将 $L_i, R_i + 1$ 放在一起离散化，这样得到的每个区间 $x_i, x_{i+1} - 1$ 就满足了我们刚刚说的情况。这时候，我们将 $f_{i,j}(x_i - 1)$ 视为已知定值，则该区间内多项式的次数仍然满足原来的性质，依旧计算 $n + 1$ 个位置的值后将区间右端点插出来，循环做就做完了。
- 分析一下复杂度，树形 dp 求解单个 T 为 $O(nk)$ ，一共有 $O(n)$ 个区间，每个区间求 $O(n)$ 个值，这一部分复杂度为 $O(n^3k)$ ；拉插部分，由于拉插的 $n + 1$ 个点横坐标连续，可以化一下拉插式子，使得其可以 $O(n)$ 求解，这样我们就只需要求解 $O(nk)$ 个多项式，各 $O(n)$ 次了。所以最终复杂度为 $O(n^3k)$ 。

卡牌养成

题意：给你一个 DAG，你要从 1 走到 n ，途中有若干特殊事件的点，有：

- 0. 无事发生；
- 1. 获得一个 (a_i, b_i) 的卡片；
- 2. 选择一个 a_i ，将其加上 x_i ；
- 3. 选择一个 b_i ，将其加上 y_i ；
- 4. 战力直接增强 w_i 。

到达 n 后选择一个卡片使 a_i 乘 10^9 。求 $\sum a_i b_i + \sum w_i$ 最大值。
 $n, a, b, x, y \leq 200, w \leq 10^6, m \leq 2000$ 。

- 注意到 $\sum a_i b_i + \sum w_i$ 怎么也凑不够 10^9 ，所以最后的乘 10^9 就是乘 ∞ 。

- 注意到 $\sum a_i b_i + \sum w_i$ 怎么也凑不够 10^9 ，所以最后的乘 10^9 就是乘 ∞ 。
- DAG 讨论起来太复杂，我们直接按链来讨论。

- 注意到 $\sum a_i b_i + \sum w_i$ 怎么也凑不够 10^9 ，所以最后的乘 10^9 就是乘 ∞ 。
- DAG 讨论起来太复杂，我们直接按链来讨论。
- 相当于我们要选择一个点 t （请记住这个 t ，后面有 call call back），作为最后乘 10^9 的那个卡， t 以后的部分非常简单，直接把所有 2 类点和 3 类点全堆给这张卡，新卡直接当一个 $w = a_i b_i$ 的 4 类点处理即可。

卡牌养成

- 注意到 $\sum a_i b_i + \sum w_i$ 怎么也凑不够 10^9 ，所以最后的乘 10^9 就是乘 ∞ 。
- DAG 讨论起来太复杂，我们直接按链来讨论。
- 相当于我们要选择一个点 t （请记住这个 t ，后面有 call call back），作为最后乘 10^9 的那个卡， t 以后的部分非常简单，直接把所有 2 类点和 3 类点全堆给这张卡，新卡直接当一个 $w = a_i b_i$ 的 4 类点处理即可。
- 而 t 以前的部分，相当于没有 $\times \infty$ 的子任务。

- 没有 3 类点的话非常简单。一个 2 类点的贡献是选择一个 j ，使最终答案增大 $x_i b_j$ 。故一定选择前缀最大的那个 b ，整个流程没有后效性，非常轻松。

- 没有 3 类点的话非常简单。一个 2 类点的贡献是选择一个 j ，使最终答案增大 $x_i b_j$ 。故一定选择前缀最大的那个 b ，整个流程没有后效性，非常轻松。
- 但是如果同时有 2 类点和 3 类点的话，这个操作就有后效性了，因为我们的贡献会多一栏 x_i 和 y_i 的相乘项。

- 但我们分析一下每个 2 类点会选择的操作位置有什么性质。假设我们能预知最后的 b_i 分别是多少的话，那 2 类点显然一定要选前缀最大的那个。也就是说，2 类点的操作位置是单调不降的，3 类点同理。

- 但我们分析一下每个 2 类点会选择的操作位置有什么性质。假设我们能预知最后的 b_i 分别是多少的话，那 2 类点显然一定要选前缀最大的那个。也就是说，2 类点的操作位置是单调不降的，3 类点同理。
- 这样我们就可以 dp 了，设 $f_{i,v_1,v_2,0/1}$ 表示 2 类点的操作点现在 $b = v_1$ ，3 类点的操作点现在 $a = v_2$ ，这两个操作点是否是同一个点，的最大战力和。

- 但我们分析一下每个 2 类点会选择的操作位置有什么性质。假设我们能预知最后的 b_i 分别是多少的话，那 2 类点显然一定要选前缀最大的那个。也就是说，2 类点的操作位置是单调不降的，3 类点同理。
- 这样我们就可以 dp 了，设 $f_{i,v_1,v_2,0/1}$ 表示 2 类点的操作点现在 $b = v_1$ ，3 类点的操作点现在 $a = v_2$ ，这两个操作点是否是同一个点，的最大战力和。
- 转移大概就是，遇到 2/3 类点按操作点更新答案，根据是否是同一个点决定是否增大 v_1/v_2 ；遇到 1 类点，答案增加 $a_i b_i$ 的同时枚举是否更新操作点；4 类点略。

- 但我们分析一下每个 2 类点会选择的操作位置有什么性质。假设我们能预知最后的 b_i 分别是多少的话，那 2 类点显然一定要选前缀最大的那个。也就是说，2 类点的操作位置是单调不降的，3 类点同理。
- 这样我们就可以 dp 了，设 $f_{i,v_1,v_2,0/1}$ 表示 2 类点的操作点现在 $b = v_1$ ，3 类点的操作点现在 $a = v_2$ ，这两个操作点是否是同一个点，的最大战力和。
- 转移大概就是，遇到 2/3 类点按操作点更新答案，根据是否是同一个点决定是否增大 v_1/v_2 ；遇到 1 类点，答案增加 $a_i b_i$ 的同时枚举是否更新操作点；4 类点略。
- 后文设 $x, y, a, b = O(V)$ 。由于 v_1, v_2 可达 $O(nV)$ 级别，直接无脑做是 $O(n^3V)$ 的，上 DAG 就是 $O(mn^2V)$ 。

卡牌养成

- 接下来是本题最重要的结论了： v_1, v_2 至少有一个不超过 V ，否则这个状态一定不可能走到最优答案。

卡牌养成

- 接下来是本题最重要的结论了： v_1, v_2 至少有一个不超过 V ，否则这个状态一定不可能走到最优答案。
- 反证。以下证明分两步：第一步是找到一个 p 使得所有操作完成后， a_p, b_p 均大于 V 。

卡牌养成

- 接下来是本题最重要的结论了： v_1, v_2 至少有一个不超过 V ，否则这个状态一定不可能走到最优答案。
- 反证。以下证明分两步：第一步是找到一个 p 使得所有操作完成后， a_p, b_p 均大于 V 。
- 考虑 v_1 对应的位置是 j ， v_2 对应的位置是 k ，如果 $j = k$ 那么 $p = j$ 就找到了；否则，不妨设 $j < k$ ，由于 $a_k = v_2 > V$ ，则一定有 2 操作对 k 操作过，那么在最终序列中一定有 $b_k \geq b_j > V$ ，则 $p = k$ 就找到了。

卡牌养成

- 接下来是本题最重要的结论了： v_1, v_2 至少有一个不超过 V ，否则这个状态一定不可能走到最优答案。
- 反证。以下证明分两步：第一步是找到一个 p 使得所有操作完成后， a_p, b_p 均大于 V 。
- 考虑 v_1 对应的位置是 j ， v_2 对应的位置是 k ，如果 $j = k$ 那么 $p = j$ 就找到了；否则，不妨设 $j < k$ ，由于 $a_k = v_2 > V$ ，则一定有 2 操作对 k 操作过，那么在最终序列中一定有 $b_k \geq b_j > V$ ，则 $p = k$ 就找到了。
- 第二步是说明 t 不比 p 优。原因很简单，前面说过 t 后面的操作都会喂给 t ，所以最终的 (a_p, b_p) 和只考虑 t 以前的操作是相等的。但是由于 $a_p > V \geq a_t, b_p > V \geq b_t$ ， t 显然不如 p ，我们直接把 t 换到 p 就得到了更优的答案，从而不需要来到 $(i, b_j = v_1, a_k = v_2)$ 这个状态。

卡牌养成

- 接下来是本题最重要的结论了： v_1, v_2 至少有一个不超过 V ，否则这个状态一定不可能走到最优答案。
- 反证。以下证明分两步：第一步是找到一个 p 使得所有操作完成后， a_p, b_p 均大于 V 。
- 考虑 v_1 对应的位置是 j ， v_2 对应的位置是 k ，如果 $j = k$ 那么 $p = j$ 就找到了；否则，不妨设 $j < k$ ，由于 $a_k = v_2 > V$ ，则一定有 2 操作对 k 操作过，那么在最终序列中一定有 $b_k \geq b_j > V$ ，则 $p = k$ 就找到了。
- 第二步是说明 t 不比 p 优。原因很简单，前面说过 t 后面的操作都会喂给 t ，所以最终的 (a_p, b_p) 和只考虑 t 以前的操作是相等的。但是由于 $a_p > V \geq a_t, b_p > V \geq b_t$ ， t 显然不如 p ，我们直接把 t 换到 p 就得到了更优的答案，从而不需要来到 $(i, b_j = v_1, a_k = v_2)$ 这个状态。
- 另外我们知道每个成为状态的 v_1 和 v_2 最终值肯定单调不降，所以不可能出现先 $v_1 > V$ 再 $v_2 > V$ 的情况。强制 $v_1 \leq V$ 跑就可以了，之后 $\text{swap}(a, b)$ 再跑一遍。复杂度降到 $O(n^2 V^2)$ 。

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。
- 我们做一个惊人的操作: 我们不再像之前一样通过分析最终的 a_i, b_i 序列来得到性质进行 dp, 我们直接预判每一个 a_i 是多少!

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。
- 我们做一个惊人的操作: 我们不再像之前一样通过分析最终的 a_i, b_i 序列来得到性质进行 dp, 我们直接预判每一个 a_i 是多少!
- 假设我们预判的 a_i 的值叫做 a'_i 。如果 $a'_i > a_i$, 那么 a'_i 为至少一个 2 操作的操作点, 所以从遇到此点开始的 2 操作都会喂给这个点。由于我们预判了每一个 a_i , 每个 3 操作的决策变得异常简单, 直接对着最大的 a'_i 操作就行。

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。
- 我们做一个惊人的操作: 我们不再像之前一样通过分析最终的 a_i, b_i 序列来得到性质进行 dp, 我们直接预判每一个 a_i 是多少!
- 假设我们预判的 a_i 的值叫做 a'_i 。如果 $a'_i > a_i$, 那么 a'_i 为至少一个 2 操作的操作点, 所以从遇到此点开始的 2 操作都会喂给这个点。由于我们预判了每一个 a_i , 每个 3 操作的决策变得异常简单, 直接对着最大的 a'_i 操作就行。
- 这样的话, 2 操作就变成了一个“还债”操作。设 $f_{i,j,k}$ 表示考虑了前 i 个点, 目前的 a 序列还有 j 的债要还, 前缀 a' 最大值为 k 。

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。
- 我们做一个惊人的操作: 我们不再像之前一样通过分析最终的 a_i, b_i 序列来得到性质进行 dp, 我们直接预判每一个 a_i 是多少!
- 假设我们预判的 a_i 的值叫做 a'_i 。如果 $a'_i > a_i$, 那么 a'_i 为至少一个 2 操作的操作点, 所以从遇到此点开始的 2 操作都会喂给这个点。由于我们预判了每一个 a_i , 每个 3 操作的决策变得异常简单, 直接对着最大的 a'_i 操作就行。
- 这样的话, 2 操作就变成了一个“还债”操作。设 $f_{i,j,k}$ 表示考虑了前 i 个点, 目前的 a 序列还有 j 的债要还, 前缀 a' 最大值为 k 。
- 转移比较简单, 1 操作要么直接更新 k , 要么从一个 $j = 0$ 的状态新起债务, 并同步更新 k , 代价是 $a'_i b_i$; 2 操作纯还债, j 减去 x_i 即可; 3 操作直接增加 $k y_i$ 的代价; 日常忽略 4 操作。

卡牌养成

- 我们知道在到达 t 之前所有的 a 都不可能超过 V (限制的), 而 b 还有 $O(nV)$ 的大小, 太区了。
- 我们做一个惊人的操作: 我们不再像之前一样通过分析最终的 a_i, b_i 序列来得到性质进行 dp, 我们直接预判每一个 a_i 是多少!
- 假设我们预判的 a_i 的值叫做 a'_i 。如果 $a'_i > a_i$, 那么 a'_i 为至少一个 2 操作的操作点, 所以从遇到此点开始的 2 操作都会喂给这个点。由于我们预判了每一个 a_i , 每个 3 操作的决策变得异常简单, 直接对着最大的 a'_i 操作就行。
- 这样的话, 2 操作就变成了一个“还债”操作。设 $f_{i,j,k}$ 表示考虑了前 i 个点, 目前的 a 序列还有 j 的债要还, 前缀 a' 最大值为 k 。
- 转移比较简单, 1 操作要么直接更新 k , 要么从一个 $j = 0$ 的状态新起债务, 并同步更新 k , 代价是 $a'_i b_i$; 2 操作纯还债, j 减去 x_i 即可; 3 操作直接增加 $k y_i$ 的代价; 日常忽略 4 操作。
- 时间复杂度降至 $O(nV^2)$, 需要枚举 a'_i 的地方可以通过前缀 max 的方式优化掉一个 V 。

- 上 DAG 的步骤是最简单的，唯一的区别是我们要对 t 到终点做一个新的 dp。

- 上 DAG 的步骤是最简单的，唯一的区别是我们要对 t 到终点做一个新的 dp。
- 记 $g_{i,j}$ 表示从 i 跑到 n ，一路上的 $\sum x_i = j$ ，将 $\sum y_i$ 作为第一关键字、 $\sum a_i b_i + \sum w_i$ 作为第二关键字的 pair 的最大值。直接做就是 $O(nmV)$ 的。

- 上 DAG 的步骤是最简单的，唯一的区别是我们要对 t 到终点做一个新的 dp。
- 记 $g_{i,j}$ 表示从 i 跑到 n ，一路上的 $\sum x_i = j$ ，将 $\sum y_i$ 作为第一关键字、 $\sum a_i b_i + \sum w_i$ 作为第二关键字的 pair 的最大值。直接做就是 $O(nmV)$ 的。
- 专门放了一档没有 1 类点就是怕有人因为这个调不出来的，不过我前面的点大概也没放这种数据，问题不大。这种情况直接跑 w 的最长路就行了。

- 上 DAG 的步骤是最简单的，唯一的区别是我们要对 t 到终点做一个新的 dp。
- 记 $g_{i,j}$ 表示从 i 跑到 n ，一路上的 $\sum x_i = j$ ，将 $\sum y_i$ 作为第一关键字、 $\sum a_i b_i + \sum w_i$ 作为第二关键字的 pair 的最大值。直接做就是 $O(nmV)$ 的。
- 专门放了一档没有 1 类点就是怕有人因为这个调不出来的，不过我前面的点大概也没放这种数据，问题不大。这种情况直接跑 w 的最长路就行了。
- 总时间复杂度 $O(mV^2 + nmV)$ 。

题意：给你一棵树和一个长为 k 的序列，让你每次选择一条边以及其一侧的连通块并将其删除，使得每次留下来的部分的点数恰好构成给定序列，问操作方案数。

$n \leq 5000, k \leq 6$ 。

- 不难想到我们枚举一个点作为根，让这个点成为最终保留下来的点之一。

- 不难想到我们枚举一个点作为根，让这个点成为最终保留下来的点之一。
- 先不管怎么计算，我们发现我们这样做必然会算重。不过好就好在对于任意方案来讲，其剩下的每一个点都会被我们算一遍，所以我们直接把最后的答案除以 s_k 就行。我们这样做有一个好处是每条边删哪边就确定了，并且祖宗边一定比子孙边删得晚。

- 不难想到我们枚举一个点作为根，让这个点成为最终保留下来的点之一。
- 先不管怎么计算，我们发现我们这样做必然会算重。不过好就好在对于任意方案来讲，其剩下的每一个点都会被我们算一遍，所以我们直接把最后的答案除以 s_k 就行。我们这样做有一个好处是每条边删哪边就确定了，并且祖宗边一定比子孙边删得晚。
- 不难发现 k 非常小，考虑我们选定删除哪条边的同时直接固定它是第几次删除的边，并把删边情况状压一下。考虑 $f_{i,s}$ 表示考虑了 i 的子树，子树内已经删除了 s 的边的方案数。

- 接下来是转移。首先合并各个子树，这个没啥说的，直接枚举 s 的子集合并就行。

- 接下来是转移。首先合并各个子树，这个没啥说的，直接枚举 s 的子集合并就行。
- 之后是判断 i 向祖先的边能否是第 j 条被删除的边，这个其实很简单。设 $\Delta_j = s_{j-1} - s_j$ ，子树内已被删除的边即使我们不知道是哪些边，我们也知道删除它们的时候删除了多少的点（利用 Δ ），所以我们就能判断这个子树内是否有恰好 Δ_j 个点。换言之，如果 j 比 s 中所有值都大（ j 是 $s|t$ 的 highbit），且 $siz_i = \Delta_j + \sum_{p \in s} \Delta_p$ ，那么这条边可以删除。

- 接下来是转移。首先合并各个子树，这个没啥说的，直接枚举 s 的子集合并就行。
- 之后是判断 i 向祖先的边能否是第 j 条被删除的边，这个其实很简单。设 $\Delta_j = s_{j-1} - s_j$ ，子树内已被删除的边即使我们不知道是哪些边，我们也知道删除它们的时候删除了多少的点（利用 Δ ），所以我们就能判断这个子树内是否有恰好 Δ_j 个点。换言之，如果 j 比 s 中所有值都大（ j 是 $s|t$ 的 highbit），且 $sz_i = \Delta_j + \sum_{p \in s} \Delta_p$ ，那么这条边可以删除。
- 故我们就得到了正确的计数方法。不难发现这个 dp 可以换根，dp 的转移是一种合并的方式，所以我们可以交换一下合并顺序（也就是前后缀合并，往下走的时候再两边合并），然后这个题就做完了。

- 接下来是转移。首先合并各个子树，这个没啥说的，直接枚举 s 的子集合并就行。
- 之后是判断 i 向祖先的边能否是第 j 条被删除的边，这个其实很简单。设 $\Delta_j = s_{j-1} - s_j$ ，子树内已被删除的边即使我们不知道是哪些边，我们也知道删除它们的时候删除了多少的点（利用 Δ ），所以我们就能判断这个子树内是否有恰好 Δ_j 个点。换言之，如果 j 比 s 中所有值都大（ j 是 $s|t$ 的 highbit），且 $sz_i = \Delta_j + \sum_{p \in s} \Delta_p$ ，那么这条边可以删除。
- 故我们就得到了正确的计数方法。不难发现这个 dp 可以换根，dp 的转移是一种合并的方式，所以我们可以交换一下合并顺序（也就是前后缀合并，往下走的时候再两边合并），然后这个题就做完了。
- 时间复杂度 $O(n3^k)$ 。

题意：给你一个序列 a ，问有多少个排列 p 满足

$$\forall i, |a_{p_i} - \max_{j < i} a_{p_j}| > k。$$

$$n \leq 10^6。$$

- 显然所有前缀最大值点是主要关注对象。设计 dp 状态时肯定和这个相关。

- 显然所有前缀最大值点是主要关注对象。设计 dp 状态时肯定和这个相关。
- 如果我们正着考虑，我们会发现我们不好确定其余每个数应该要插到哪里去。考虑反着做， f_i 表示考虑了 $i \sim n$ 的所有数，目前填好的数满足前面的条件。

- 显然所有前缀最大值点是主要关注对象。设计 dp 状态时肯定和这个相关。
- 如果我们正着考虑，我们会发现我们不好确定其余每个数应该要插到哪里去。考虑反着做， f_i 表示考虑了 $i \sim n$ 的所有数，目前填好的数满足前面的条件。
- 转移的话，如果我们新填入的一个数 $i - 1$ 不放在开头，我们这边暂时无脑将 $i - 1$ 插入到后面 $n - i + 1$ 个位置中，至于为什么我们这边无脑插入，我们直接把所有可能发生问题的数在第二种情况里一起转移了。

- 显然所有前缀最大值点是主要关注对象。设计 dp 状态时肯定和这个相关。
- 如果我们正着考虑，我们会发现我们不好确定其余每个数应该要插到哪里去。考虑反着做， f_i 表示考虑了 $i \sim n$ 的所有数，目前填好的数满足前面的条件。
- 转移的话，如果我们新填入的一个数 $i - 1$ 不放在开头，我们这边暂时无脑将 $i - 1$ 插入到后面 $n - i + 1$ 个位置中，至于为什么我们这边无脑插入，我们直接把所有可能发生问题的数在第二种情况里一起转移了。
- 第二种转移就是 $i - 1$ 放在开头，那么我们首先要把 $l_{i-1} \sim i - 2$ 里的所有数全部插入 $i - 1$ 后面去，其中 l_{i-1} 是大于等于 $a_{i-1} - k$ 的最左边的数。这个的方案数显然用一串连乘就可以搞定，预处理一下阶乘，这里就是一个 A 的系数。也就是说，这里是直接从 f_i 贡献到 $f_{l_{i-1}}$ 去了。

- 显然所有前缀最大值点是主要关注对象。设计 dp 状态时肯定和这个相关。
- 如果我们正着考虑，我们会发现我们不好确定其余每个数应该要插到哪里去。考虑反着做， f_i 表示考虑了 $i \sim n$ 的所有数，目前填好的数满足前面的条件。
- 转移的话，如果我们新填入的一个数 $i - 1$ 不放在开头，我们这边暂时无脑将 $i - 1$ 插入到后面 $n - i + 1$ 个位置中，至于为什么我们这边无脑插入，我们直接把所有可能发生问题的数在第二种情况里一起转移了。
- 第二种转移就是 $i - 1$ 放在开头，那么我们首先要把 $l_{i-1} \sim i - 2$ 里的所有数全部插入 $i - 1$ 后面去，其中 l_{i-1} 是大于等于 $a_{i-1} - k$ 的最左边的数。这个的方案数显然用一串连乘就可以搞定，预处理一下阶乘，这里就是一个 A 的系数。也就是说，这里是直接从 f_i 贡献到 $f_{l_{i-1}}$ 去了。
- 这样我们的 dp 就没有问题了。时间复杂度 $O(n)$ 。

题意：给你一个初始均为 0 的序列，之后每次给一个 0 赋值，使得最终是一个排列。问你每次赋值之后，共有多少种你来接手后续赋值的方式，使得赋值之后将这个排列视为一个置换，所有的置换环长度都是 3 的倍数。

$$n \leq 3 \times 10^5。$$

- 显然每个链具体是多少我们是不用管的，我们只关心有没有已经不是 3 的倍数的环，以及有多少模 3 长度分别是 1, 2, 0 的链。

- 显然每个链具体是多少我们是不用管的，我们只关心有没有已经不是 3 的倍数的环，以及有多少模 3 长度分别是 1, 2, 0 的链。
- 设这些链共有 a, b, c 个，设方案数为 $f_{a,b,c}$ 。注意到我们可以挑选包含最小编号的那条 0 的链，把它挂到任意一个其他链的后面，或直接自己成环，则有 $f_{a,b,c} = (a + b + c)f_{a,b,c-1}$ 。我们只需要计算 $g_{a,b} = f_{a,b,0}$ 即可。

- 显然每个链具体是多少我们是不用管的，我们只关心有没有已经不是 3 的倍数的环，以及有多少模 3 长度分别是 1, 2, 0 的链。
- 设这些链共有 a, b, c 个，设方案数为 $f_{a,b,c}$ 。注意到我们可以挑选包含最小编号的那条 0 的链，把它挂到任意一个其他链的后面，或直接自己成环，则有 $f_{a,b,c} = (a + b + c)f_{a,b,c-1}$ 。我们只需要计算 $g_{a,b} = f_{a,b,0}$ 即可。
- 注意到就算是 $g_{a,b}$ 我们也没法求。不过 $g_{0,0}$ 是很好算的，我们不妨假设最终的排列是有解的，如果无解我们可以从出现无解的那一刻开始抛弃掉原有排列，自行将剩余的所有链一一穿起来，只不过最后输出的时候直接输出无解就行。

- 显然每个链具体是多少我们是不用管的，我们只关心有没有已经不是 3 的倍数的环，以及有多少模 3 长度分别是 1, 2, 0 的链。
- 设这些链共有 a, b, c 个，设方案数为 $f_{a,b,c}$ 。注意到我们可以挑选包含最小编号的那条 0 的链，把它挂到任意一个其他链的后面，或直接自己成环，则有 $f_{a,b,c} = (a + b + c)f_{a,b,c-1}$ 。我们只需要计算 $g_{a,b} = f_{a,b,0}$ 即可。
- 注意到就算是 $g_{a,b}$ 我们也没法求。不过 $g_{0,0}$ 是很好算的，我们不妨假设最终的排列是有解的，如果无解我们可以从出现无解的那一刻开始抛弃掉原有排列，自行将剩余的所有链一一穿起来，只不过最后输出的时候直接输出无解就行。
- 在这样的情况下，我们相当于要进行多次 (a, b) 的单点求值，容易发现两次相邻查询的 (a, b) 差距不会太大。考虑递推。

- 正着来显然不对，直接看反着来，如果 (a, b) 发生变化，那肯定是两条 1 合并、两条 2 合并、一个 1 一个 2 合并三种情况之一，其前一个询问的 (a, b) 只会有 $(a + 2, b - 1)$, $(a - 1, b + 2)$, $(a + 1, b + 1)$ 三种情况。

- 正着来显然不对，直接看反着来，如果 (a, b) 发生变化，那肯定是两条 1 合并、两条 2 合并、一个 1 一个 2 合并三种情况之一，其前一个询问的 (a, b) 只会有 $(a + 2, b - 1)$, $(a - 1, b + 2)$, $(a + 1, b + 1)$ 三种情况。
- 接下来我们只要写出 $g_{a,b}$ 的递推式，我们大概率就可以通过递推式来递推出每组询问的解了。我们考虑包含最小点的 1，其出边有 b 种选到 2，这样就变成了 $f_{a-1,b-1,1} = (a + b - 1)g_{a-1,b-1}$ ；有 $a - 1$ 种选到 1，这样就变成了 $g_{a-2,b+1}$ 。同理考虑包含最小点的 2，则：

$$g_{a,b} = b(a + b - 1)g_{a-1,b-1} + (a - 1)g_{a-2,b+1}$$

$$g_{a,b} = a(a + b - 1)g_{a-1,b-1} + (b - 1)g_{a+1,b-2}$$

- 正着来显然不对，直接看反着来，如果 (a, b) 发生变化，那肯定是两条 1 合并、两条 2 合并、一个 1 一个 2 合并三种情况之一，其前一个询问的 (a, b) 只会有 $(a + 2, b - 1), (a - 1, b + 2), (a + 1, b + 1)$ 三种情况。
- 接下来我们只要写出 $g_{a,b}$ 的递推式，我们大概率就可以通过递推式来递推出每组询问的解了。我们考虑包含最小点的 1，其出边有 b 种选到 2，这样就变成了 $f_{a-1,b-1,1} = (a + b - 1)g_{a-1,b-1}$ ；有 $a - 1$ 种选到 1，这样就变成了 $g_{a-2,b+1}$ 。同理考虑包含最小点的 2，则：

$$g_{a,b} = b(a + b - 1)g_{a-1,b-1} + (a - 1)g_{a-2,b+1}$$

$$g_{a,b} = a(a + b - 1)g_{a-1,b-1} + (b - 1)g_{a+1,b-2}$$

- 分析出了这个递推式后，考虑维护 $X = g_{a,b}, Y = g_{a+1,b+1}$ ，剩下的部分就可以通过上面的递推式进行递推了。注意 $(a, b) \rightarrow (a + 1, b + 1)$ 可以看作 $(a, b) \rightarrow (a + 2, b - 1) \rightarrow ((a + 2) - 1, (b - 1) + 2)$ ，这题就做完了。

- 正着来显然不对，直接看反着来，如果 (a, b) 发生变化，那肯定是两条 1 合并、两条 2 合并、一个 1 一个 2 合并三种情况之一，其前一个询问的 (a, b) 只会有 $(a + 2, b - 1), (a - 1, b + 2), (a + 1, b + 1)$ 三种情况。
- 接下来我们只要写出 $g_{a,b}$ 的递推式，我们大概率就可以通过递推式来递推出每组询问的解了。我们考虑包含最小点的 1，其出边有 b 种选到 2，这样就变成了 $f_{a-1,b-1,1} = (a + b - 1)g_{a-1,b-1}$ ；有 $a - 1$ 种选到 1，这样就变成了 $g_{a-2,b+1}$ 。同理考虑包含最小点的 2，则：

$$g_{a,b} = b(a + b - 1)g_{a-1,b-1} + (a - 1)g_{a-2,b+1}$$

$$g_{a,b} = a(a + b - 1)g_{a-1,b-1} + (b - 1)g_{a+1,b-2}$$

- 分析出了这个递推式后，考虑维护 $X = g_{a,b}, Y = g_{a+1,b+1}$ ，剩下的部分就可以通过上面的递推式进行递推了。注意 $(a, b) \rightarrow (a + 1, b + 1)$ 可以看作 $(a, b) \rightarrow (a + 2, b - 1) \rightarrow ((a + 2) - 1, (b - 1) + 2)$ ，这题就做完了。
- 时间复杂度 $O(n + n\alpha(n))$ ，其中 α 为并查集复杂度。

题意：给你一个 01 序列，你必须进行恰好 k 次交换操作，每次选择相邻的 01 或者 10，并将其交换，问最后能形成多少种不同的序列。
 $n \leq 1500, k \leq 1500$ 。

- 比较无脑的想法就是直接对着最终序列 dp, 考虑 $f_{i,j,k}$ 表示现在考虑完了最终序列的前 i 个位置, 目前已经移动了 j 步, 前面已经有 k 个 1 的方案数。

- 比较无脑的想法就是直接对着最终序列 dp, 考虑 $f_{i,j,k}$ 表示现在考虑完了最终序列的前 i 个位置, 目前已经移动了 j 步, 前面已经有 k 个 1 的方案数。
- 转移应该是简单的, 考虑下个位置放不放 1, 放 1 的话就需要 $|p_{k+1} - (i - 1)|$ 次交换来将第 $k + 1$ 个 1 交换过来, 这里 p_{k+1} 指的是原序列第 $k + 1$ 个 1 的位置。

- 比较无脑的想法就是直接对着最终序列 dp, 考虑 $f_{i,j,k}$ 表示现在考虑完了最终序列的前 i 个位置, 目前已经移动了 j 步, 前面已经有 k 个 1 的方案数。
- 转移应该是简单的, 考虑下个位置放不放 1, 放 1 的话就需要 $|p_{k+1} - (i - 1)|$ 次交换来将第 $k + 1$ 个 1 交换过来, 这里 p_{k+1} 指的是原序列第 $k + 1$ 个 1 的位置。
- 但这个 dp 是 $O(n^2k)$ 的, 爆炸了。

- 发现我们没有更多能够减少状态维度的方法了，我们去考察一下状态数。

- 发现我们没有更多能够减少状态维度的方法了，我们去考察一下状态数。
- 注意到这样一件事情：我们的交换次数其实偏少。如果前 i 个位置比原有序列少了太多 1 或者多了太多 1，为了形成这样的局面，我们需要把 1 交换出去或者交换进来，而最近的至少要 1 次交换，次近的至少要 2 次，第三近的至少要 3 次等等。故我们发现这样一个特点： k 实际上只有和原序列 1 的个数相差不到 $O(\sqrt{k})$ 种取值。

- 发现我们没有更多能够减少状态维度的方法了，我们去考察一下状态数。
- 注意到这样一件事情：我们的交换次数其实偏少。如果前 i 个位置比原有序列少了太多 1 或者多了太多 1，为了形成这样的局面，我们需要把 1 交换出去或者交换进来，而最近的至少要 1 次交换，次近的至少要 2 次，第三近的至少要 3 次等等。故我们发现这样一个特点： k 实际上只有和原序列 1 的个数相差不到 $O(\sqrt{k})$ 种取值。
- 这样复杂度降到了 $O(nk\sqrt{k})$ ，就过了。

题意：你在做快排，但是你每次都选择区间左端点作为分割数。设你取区间左端点 x 为分割数，将序列按照原顺序分割成了小于 x 的 L 和大于 x 的 R ，定义 $F(A)$ 为一个序列的操作次数，则 $F(A) = |A| + F(L) + F(R)$ 。求有多少个排列 p 使得 $F(p) \geq \text{lim}$ 。
 $n \leq 200, \text{lim} \leq 10^9$ 。

- 专门把 $\lim$ 写出来就是为了吓你一跳让你不敢往状态里写的 (x

- 专门把 lim 写出来就是为了吓你一跳让你不敢往状态里写的 (x
- 实际上 lim 显然小于 n^2 , 所以 lim 太大了直接 0 就行。

- 专门把 lim 写出来就是为了吓你一跳让你不敢往状态里写的 (×)
- 实际上 lim 显然小于 n^2 , 所以 lim 太大了直接 0 就行。
- 非常粗暴地设 $f_{x,y}$ 表示长度为 x 的序列有多少个恰好 $F = y$ 的排列。转移直接枚举 L 的大小 a 和 $F = b$, 乘上一个左右合并系数, 则有:

$$f_{x,y} = \sum_{a=0}^{x-1} C_{x-1}^a \sum_{b=0}^{y-x} f_{a,b} f_{x-a-1,y-x-b}$$

不过 y 是 $O(n^2)$ 级别的, 直接做 $O(n^6)$, 飞起来了。

- 如果我们固定 a ，我们看后面那部分，其实就是 $f_{i,*}$ 和 $f_{j,*}$ 的背包罢了。NTT 还是太弱了， $O(n^4 \log n)$ 自带大常数肯定过不了。

- 如果我们固定 a ，我们看后面那部分，其实就是 $f_{i,*}$ 和 $f_{j,*}$ 的背包罢了。NTT 还是太弱了， $O(n^4 \log n)$ 自带大常数肯定过不了。
- 但我们知道背包可以干这么一件事情：考虑一个关于 z 的多项式函数 $F_x(z) = \sum_{i=0}^{\infty} f_{x,i} z^i$ ，那右边那玩意其实就是 F_a 和 F_{x-a-1} 相乘后多项式 z^{y-x} 项前面的系数罢了。我们要求的也是所有的 $f_{n,y}$ ，也就是多项式 $F_n(z)$ 长什么样子。

- 如果我们固定 a ，我们看后面那部分，其实就是 $f_{i,*}$ 和 $f_{j,*}$ 的背包罢了。NTT 还是太弱了， $O(n^4 \log n)$ 自带大常数肯定过不了。
- 但我们知道背包可以干这么一件事情：考虑一个关于 z 的多项式函数 $F_x(z) = \sum_{i=0}^{\infty} f_{x,i} z^i$ ，那右边那玩意其实就是 F_a 和 F_{x-a-1} 相乘后多项式 z^{y-x} 项前面的系数罢了。我们要求的也是所有的 $f_{n,y}$ ，也就是多项式 $F_n(z)$ 长什么样子。
- 我们可以干这么一个逆天的事情：我们直接选择一个真正的数 z ，计算出 $F_n(z)$ 的值！这样我们如果能求出 $O(n^2)$ 个值，我们就可以把最终多项式直接插出来，这样求个系数肯定是轻轻松松。

- 原式按生成函数写出来就是：

$$F_x(z) = \sum_{a=0}^{x-1} C_{x-1}^a z^x F_a(z) F_{x-a-1}(z),$$
 之所以要乘一个 z^x 是因为右边那玩意是 z^{y-x} 的系数，而左边是 z^y 的系数，差了 z^x 倍。

- 原式按生成函数写出来就是：

$$F_x(z) = \sum_{a=0}^{x-1} C_{x-1}^a z^x F_a(z) F_{x-a-1}(z),$$
 之所以要乘一个 z^x 是因为右边那玩意是 z^{y-x} 的系数，而左边是 z^y 的系数，差了 z^x 倍。

- 带入 $z = 1 \sim n^2$ ，分别依照这个式子 $O(n^2)$ 地求出 $F_n(z)$ ，最后利用拉格朗日插值的定义式展开就可以通过了。

- 原式按生成函数写出来就是：

$$F_x(z) = \sum_{a=0}^{x-1} C_{x-1}^a z^x F_a(z) F_{x-a-1}(z), \text{ 之所以要乘一个 } z^x \text{ 是因为}$$

右边那玩意是 z^{y-x} 的系数，而左边是 z^y 的系数，差了 z^x 倍。

- 带入 $z = 1 \sim n^2$ ，分别依照这个式子 $O(n^2)$ 地求出 $F_n(z)$ ，最后利用拉格朗日插值的定义式展开就可以通过了。
- 不过展开拉插的时候如果暴力多项式乘法的话复杂度会达到五次方（求和二次方，二次方个一次式相乘，每次多项式乘法一次方）。发现对每次求和，右边那个连乘的分子都长得很像，只缺一项。那我们先把所有的都乘起来，缺的那一项再把多项式乘法当作背包撤销撤回就行了，分母是常数可以暴力，然后就做完了。

- 原式按生成函数写出来就是：

$$F_x(z) = \sum_{a=0}^{x-1} C_{x-1}^a z^x F_a(z) F_{x-a-1}(z),$$
 之所以要乘一个 z^x 是因为右边那玩意是 z^{y-x} 的系数，而左边是 z^y 的系数，差了 z^x 倍。

- 带入 $z = 1 \sim n^2$ ，分别依照这个式子 $O(n^2)$ 地求出 $F_n(z)$ ，最后利用拉格朗日插值的定义式展开就可以通过了。
- 不过展开拉插的时候如果暴力多项式乘法的话复杂度会达到五次方（求和二次方，二次方个一次式相乘，每次多项式乘法一次方）。发现对每次求和，右边那个连乘的分子都长得很像，只缺一项。那我们先把所有的都乘起来，缺的那一项再把多项式乘法当作背包撤销撤回就行了，分母是常数可以暴力，然后就做完了。
- 复杂度 $O(n^4)$ 。

题意：给定长为 n 的数组 a_i ，称一个 $1 \sim n$ 的排列 p 不是好的，当且仅当存在一个区间 (l, r) ，满足 $r \leq a_l$ 且 $p_l \sim p_r$ 是一个 $l \sim r$ 的排列。问有多少个 $1 \sim n$ 的排列是好的。
 $n \leq 200$ 。

- 要求每个题给区间的子区间都不是排列，先给它容斥了。也就是说，暴力的思路是我们钦定一个区间集合 $S \subseteq U = \{[l, r] | r \leq a_l\}$ ，强制这个集合内的区间为对应排列，求出其方案数，然后用 $(-1)^{|S|}$ 的系数加起来。

- 要求每个题给区间的子区间都不是排列，先给它容斥了。也就是说，暴力的思路是我们钦定一个区间集合 $S \subseteq U = \{[l, r] | r \leq a_l\}$ ，强制这个集合内的区间为对应排列，求出其方案数，然后用 $(-1)^{|S|}$ 的系数加起来。
- 但是对于这种错综复杂的区间关系我们实在是无从下手，直到我们发现这个题明明可以直接给出若干个不同的区间构成集合 U ，却选择了使用这种有一定性质的给区间方式。

- 要求每个题给区间的子区间都不是排列，先给它容斥了。也就是说，暴力的思路是我们钦定一个区间集合 $S \subseteq U = \{[l, r] | r \leq a_l\}$ ，强制这个集合内的区间为对应排列，求出其方案数，然后用 $(-1)^{|S|}$ 的系数加起来。
- 但是对于这种错综复杂的区间关系我们实在是无从下手，直到我们发现这个题明明可以直接给出若干个不同的区间构成集合 U ，却选择了使用这种有一定性质的给区间方式。
- 从另一个角度想，我们完全无法考虑的区间关系其实就是相交但不包含，我们考虑利用容斥与 U 的性质，构造双射，看能不能把这种抽象情况扔掉。

- 要求每个题给区间的子区间都不是排列，先给它容斥了。也就是说，暴力的思路是我们钦定一个区间集合 $S \subseteq U = \{[l, r] | r \leq a_l\}$ ，强制这个集合内的区间为对应排列，求出其方案数，然后用 $(-1)^{|S|}$ 的系数加起来。
- 但是对于这种错综复杂的区间关系我们实在是无从下手，直到我们发现这个题明明可以直接给出若干个不同的区间构成集合 U ，却选择了使用这种有一定性质的给区间方式。
- 从另一个角度想，我们完全无法考虑的区间关系其实就是相交但不包含，我们考虑利用容斥与 U 的性质，构造双射，看能不能把这种抽象情况扔掉。
- 然后我们发现可以。假设两个区间 $l < L \leq r < R$ 同时在 S 中，则我们发现区间 $[l, L-1] \in U$ ，且根据题设我们可以轻松推出把这个区间加入 S 后，方案数不变。于是我们取出最靠左的 (l, L, r, R) 对，反转 $[l, L-1]$ 在 S 中的存在情况，我们就得到了一组双射。

- 要求每个题给区间的子区间都不是排列，先给它容斥了。也就是说，暴力的思路是我们钦定一个区间集合 $S \subseteq U = \{[l, r] | r \leq a_l\}$ ，强制这个集合内的区间为对应排列，求出其方案数，然后用 $(-1)^{|S|}$ 的系数加起来。
- 但是对于这种错综复杂的区间关系我们实在是无从下手，直到我们发现这个题明明可以直接给出若干个不同的区间构成集合 U ，却选择了使用这种有一定性质的给区间方式。
- 从另一个角度想，我们完全无法考虑的区间关系其实就是相交但不包含，我们考虑利用容斥与 U 的性质，构造双射，看能不能把这种抽象情况扔掉。
- 然后我们发现可以。假设两个区间 $l < L \leq r < R$ 同时在 S 中，则我们发现区间 $[l, L-1] \in U$ ，且根据题设我们可以轻松推出把这个区间加入 S 后，方案数不变。于是我们取出最靠左的 (l, L, r, R) 对，反转 $[l, L-1]$ 在 S 中的存在情况，我们就得到了一组双射。
- 接下来只需要考虑区间要么包含要么相离的情况了。

- 区间包含与相离关系很像一棵树，并且我们需要考虑的“子树”根节点是一个个的区间，所以很容易想到区间 dp。大手一挥，设 $f_{l,r}$ 表示 $[l, r] \in S$ ，仅考虑该区间内的所有区间的情况下方案数的加权总和。注意到完全无法转移，需要辅助转移。

- 区间包含与相离关系很像一棵树，并且我们需要考虑的“子树”根节点是一个个的区间，所以很容易想到区间 dp。大手一挥，设 $f_{l,r}$ 表示 $[l, r] \in S$ ，仅考虑该区间内的所有区间的情况下方案数的加权总和。注意到完全无法转移，需要辅助转移。
- 我们考虑树意义下 $[l, r]$ 的每个儿子，我们无法转移的主要原因是不知道这些儿子中间空了多少，因为空了多少最后需要乘一个阶乘来分配。

- 区间包含与相离关系很像一棵树，并且我们需要考虑的“子树”根节点是一个个的区间，所以很容易想到区间 dp。大手一挥，设 $f_{l,r}$ 表示 $[l, r] \in S$ ，仅考虑该区间内的所有区间的情况下方案数的加权总和。注意到完全无法转移，需要辅助转移。
- 我们考虑树意义下 $[l, r]$ 的每个儿子，我们无法转移的主要原因是不知道这些儿子中间空了多少，因为空了多少最后需要乘一个阶乘来分配。
- 所以我们定义 $g_{l,r,k}$ 表示以 l 为左端点，现在已经考虑了前面的所有儿子，目前已经考虑了 r 之前的位置，没有被任何一个儿子覆盖的位置有 k 个。转移有两种，一种是 r 没有被覆盖，一种是 r 被覆盖了，枚举这个儿子的左端点是谁。 f 的转移也简单了，枚举 k 用 $g_{l,r,k}$ 转移即可。

- 区间包含与相离关系很像一棵树，并且我们需要考虑的“子树”根节点是一个个的区间，所以很容易想到区间 dp。大手一挥，设 $f_{l,r}$ 表示 $[l, r] \in S$ ，仅考虑该区间内的所有区间的情况下方案数的加权总和。注意到完全无法转移，需要辅助转移。
- 我们考虑树意义下 $[l, r]$ 的每个儿子，我们无法转移的主要原因是不知道这些儿子中间空了多少，因为空了多少最后需要乘一个阶乘来分配。
- 所以我们定义 $g_{l,r,k}$ 表示以 l 为左端点，现在已经考虑了前面的所有儿子，目前已经考虑了 r 之前的位置，没有被任何一个儿子覆盖的位置有 k 个。转移有两种，一种是 r 没有被覆盖，一种是 r 被覆盖了，枚举这个儿子的左端点是谁。 f 的转移也简单了，枚举 k 用 $g_{l,r,k}$ 转移即可。
- 实现略有细节，复杂度 $O(n^4)$ 看似无法通过且优化空间极少，但是由于区间 dp 自带超小常数，跑的飞快。

- 区间包含与相离关系很像一棵树，并且我们需要考虑的“子树”根节点是一个个的区间，所以很容易想到区间 dp。大手一挥，设 $f_{l,r}$ 表示 $[l, r] \in S$ ，仅考虑该区间内的所有区间的情况下方案数的加权总和。注意到完全无法转移，需要辅助转移。
- 我们考虑树意义下 $[l, r]$ 的每个儿子，我们无法转移的主要原因是不知道这些儿子中间空了多少，因为空了多少最后需要乘一个阶乘来分配。
- 所以我们定义 $g_{l,r,k}$ 表示以 l 为左端点，现在已经考虑了前面的所有儿子，目前已经考虑了 r 之前的位置，没有被任何一个儿子覆盖的位置有 k 个。转移有两种，一种是 r 没有被覆盖，一种是 r 被覆盖了，枚举这个儿子的左端点是谁。 f 的转移也简单了，枚举 k 用 $g_{l,r,k}$ 转移即可。
- 实现略有细节，复杂度 $O(n^4)$ 看似无法通过且优化空间极少，但是由于区间 dp 自带超小常数，跑的飞快。
- bonus：其实本来今天 t3 打算放明天的，但我注意到今天 t3 和这两道题在同一场 cf，所以紧急 swap 了一下。